Network Security

Demonstrate Vulnerability Testing on a Web Application

Introduction

This lab is designed to work on the Kali VM and Metasploitable VM to understand how web applications communicate using HTTP methods and how security vulnerabilities can be identified through interception and analysis of web traffic. Lab started by confirming Burp Suite has installed on Kali VM. Then use Burp Suite to monitor, intercept, and manipulate HTTP requests and responses between a client and a web server. Damn Vulnerable Web Application (DVWA) which hosted on the Metasploitable VM was used as the testing environment.

First, Burp Suite was configured as a proxy to capture web traffic from the Firefox browser. This will allow us to monitor detailed inspection of HTTP methods such as GET and POST. Also, we can check parameters transmitted during login attempts. The DVWA security level was set to Low to minimize protections and enable effective vulnerability testing.

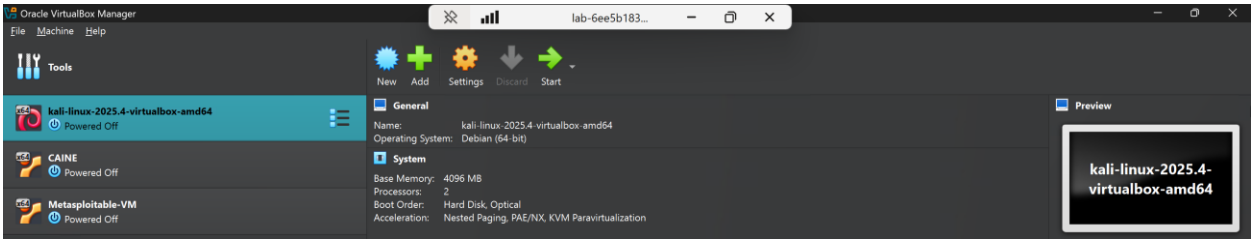
Then a brute force attack was conducted by submitting multiple usernames and passwords combinations to identify valid credentials using Burp Suite intruder tool. This lab shows how improper authentication mechanisms can be exploited and highlights the importance of secure coding practices.

Part 1 - Burp Suite

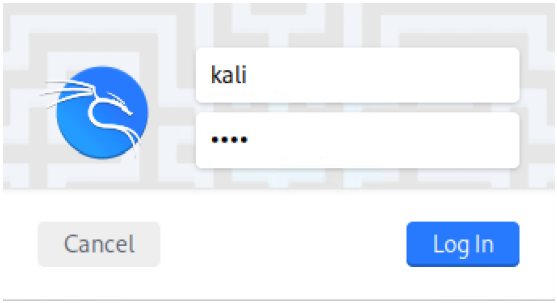
Step1 - Verifying you have Burp Suite Installed

1. Logged to Azure Lab environment using following web link and logged into the Kali VM using Oracle VirtualBox.
https://labs.azure.com/virtualmachines?feature_vnext=true

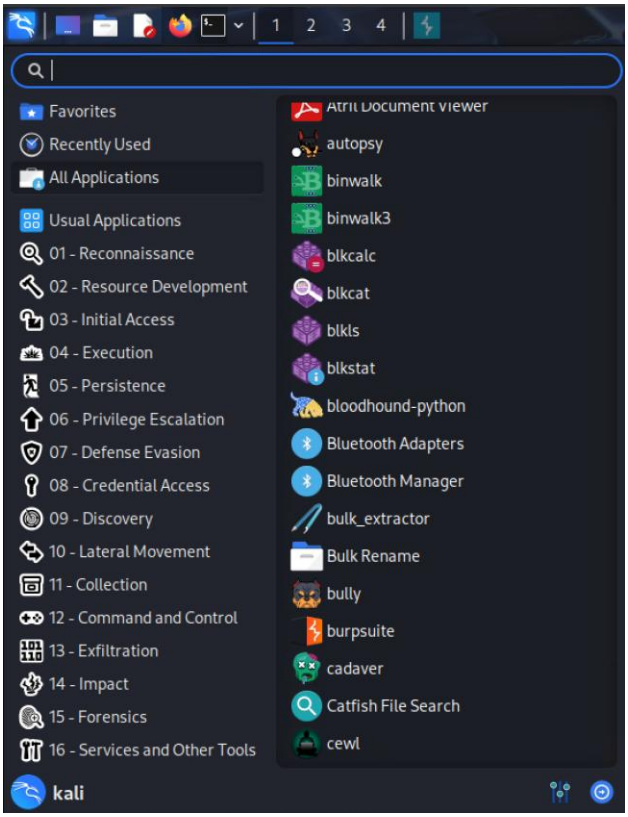
2. Under the CSIS folder, located and run Oracle Virtualbox to check the availability of Kali VM.



3. Logged to the Kali VM using the credentials '*kali'* and '*kali'* as username and password.

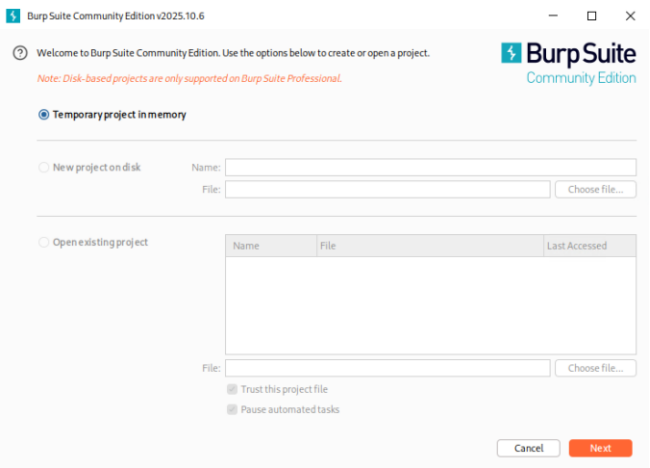


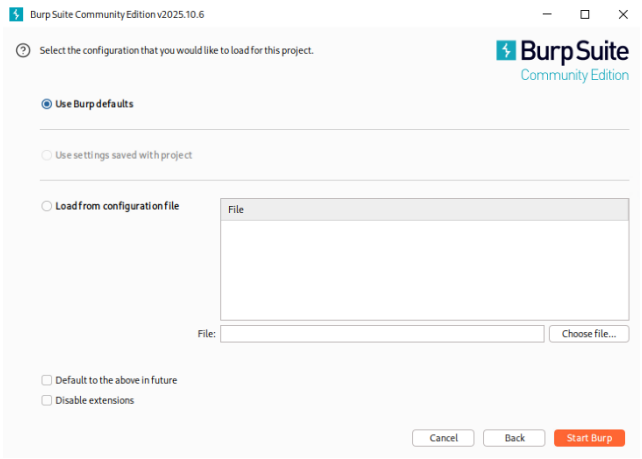
4. Used the search bar to search for Burp Suite and verified that Burp Suite was installed.



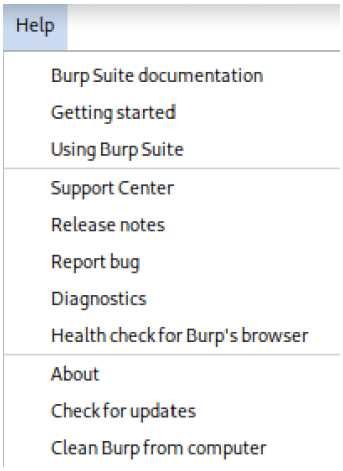
Step 2 – Using Burp Suite for Vulnerability Assessment

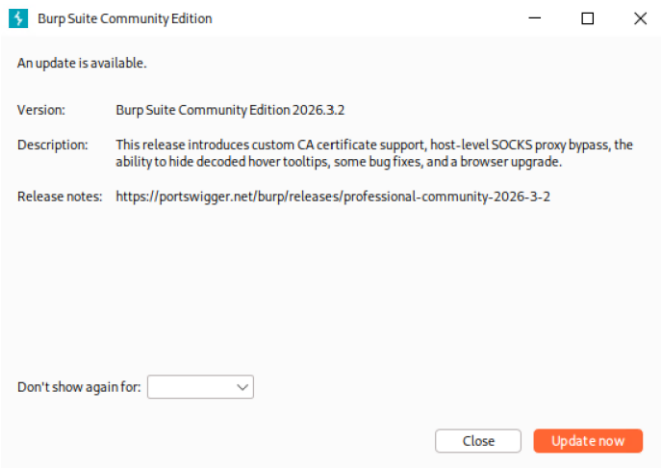
1. Opened Burp Suite. For the '*JRE*' message, clicked 'OK', accepted '*Temporary Project in Memory'*, and clicked Next. Accept to use Burp defaults. Then clicked '*Start Burp'*.



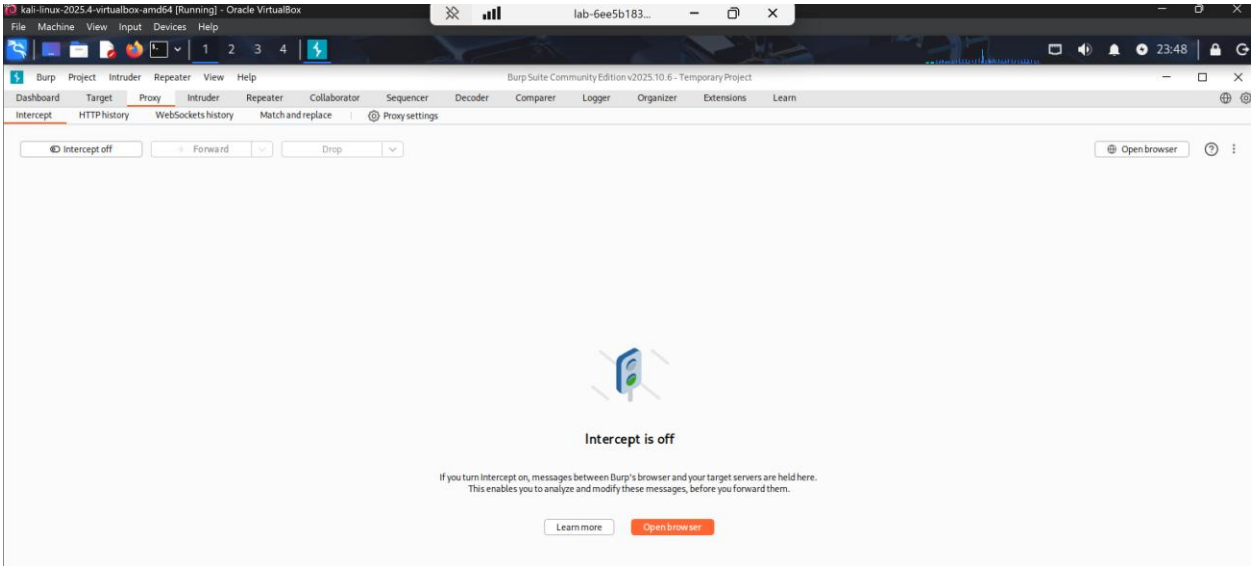


2. We can update Burp by navigating to the Help Menu > Check for updates. Then clicked Update now. But I skipped the update.

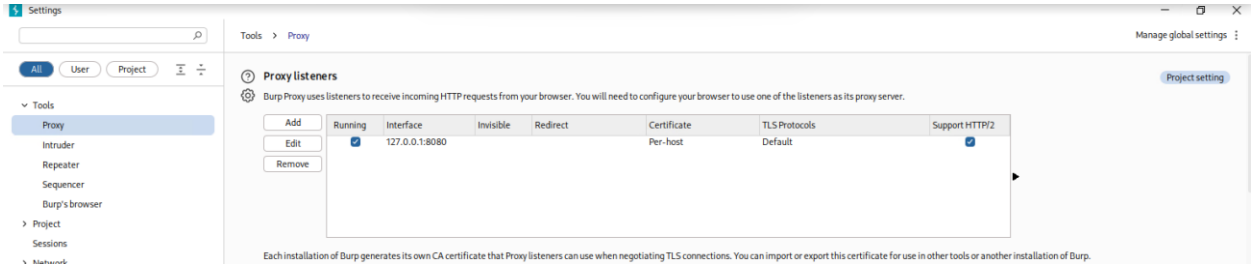




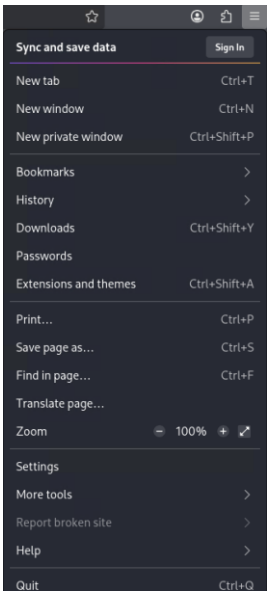
3. Cliked on Proxy.



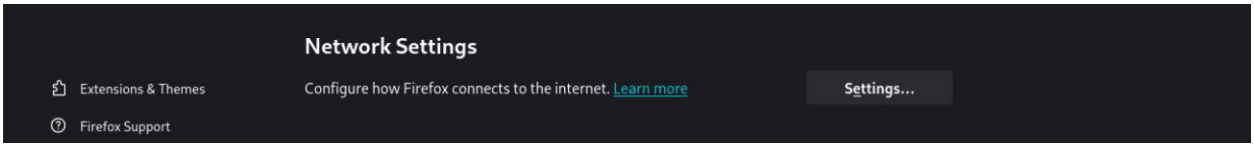
4. Clicked on *Proxy setting*s to see information on the settings (127.0.0.1:8080). Then closed the Burp Suite.



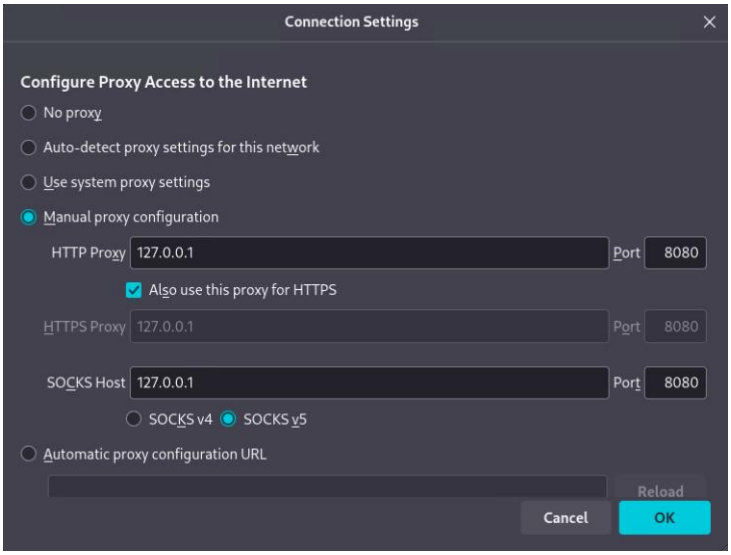
5. To configure Firefox as a proxy. Firefox opened within the Kali VM to set a proxy inside the browser. Then I clicked the three bars on the right and scrolled to the bottom and clicked on *Settings*.



6. Scrolled down to *Network Settings*, and there, clicked on *Settings.*

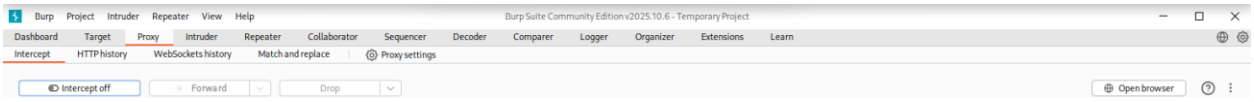


7. Configured the proxy as follows and clicked *OK*.

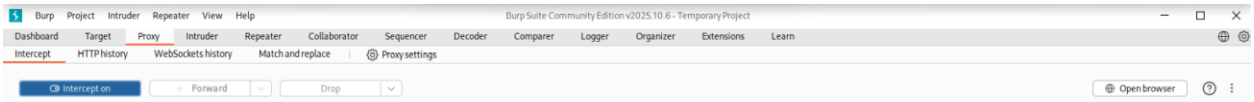


8. Reopened Burp Suite by accepting defaults.

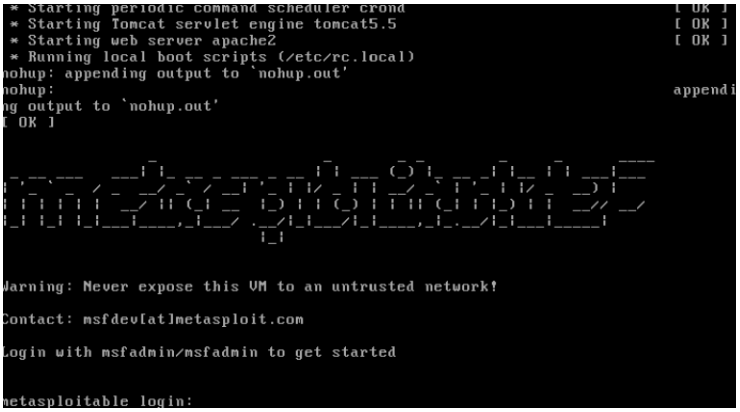
9. Clicked on *Proxy* and then clicked on *Intercept*.



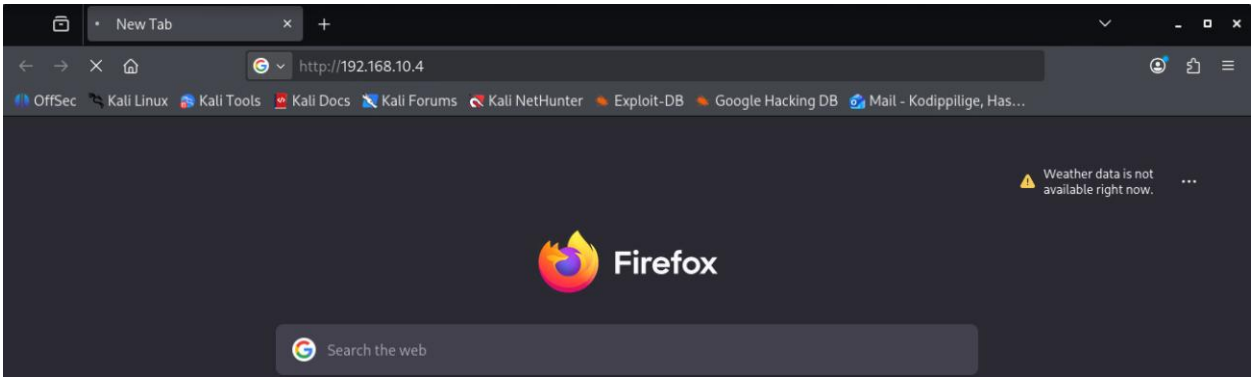
10. Clicked on the *Intercept is off* button to turn *Intercept on*.



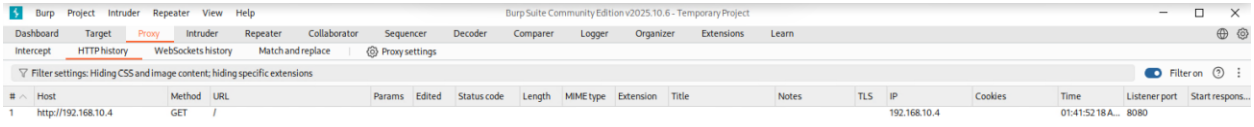
11. To see if Burp Suite is acting as a proxy, started the Metasploitable VM in Oracle VirtualBox.



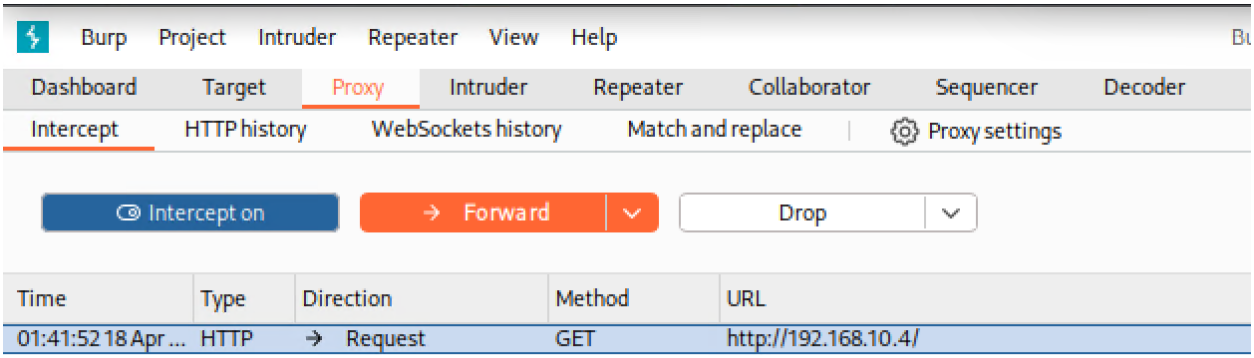
12. In Kali, I used Firefox browser to go to the IP address of the Metasploitable VM by typing 192.168.10.4 on address bar (The URL I entered in Firefox will not open a web page. Burp will however capture the traffic).

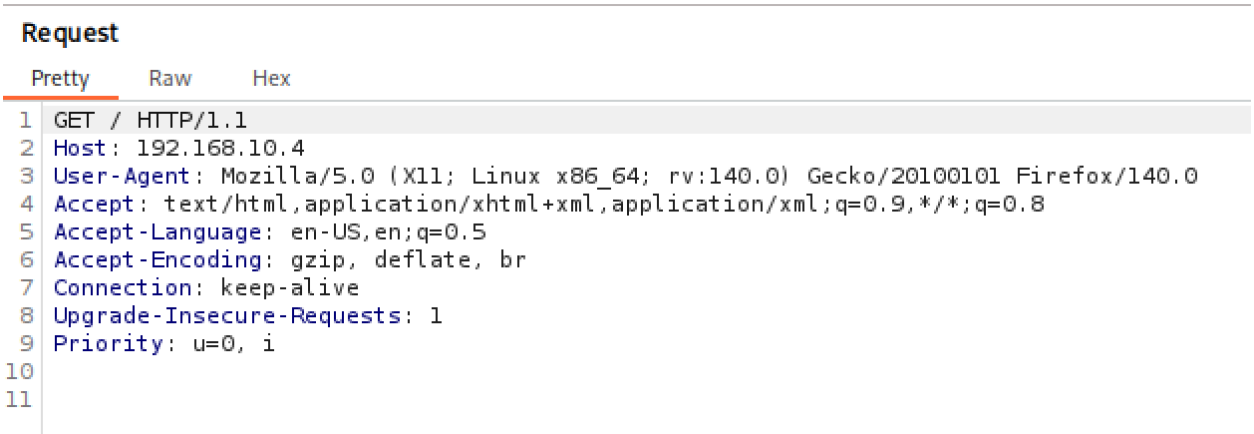


13. Clicked the *HTTP history* tab on Burp Suite. It shows that Metaspoitable VM IP address, it means Burp suite has successfully intercepted traffic, as a proxy.

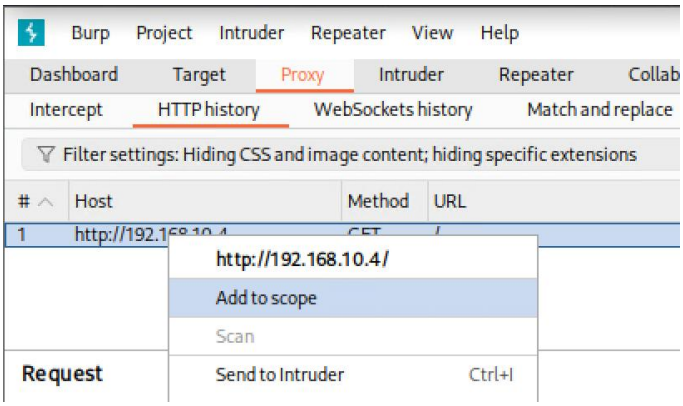


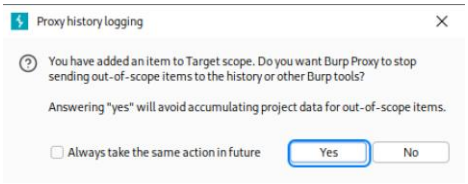
14. Clicked on Intercept tab to get more detailed information.





15. Next, I set up the scope. Clicked on HTTP history tab and right clicked on the target URL. Then I clicked on *Add to Scope*. Clicked '*Yes*' in the 'Proxy history' dialog box that opens.

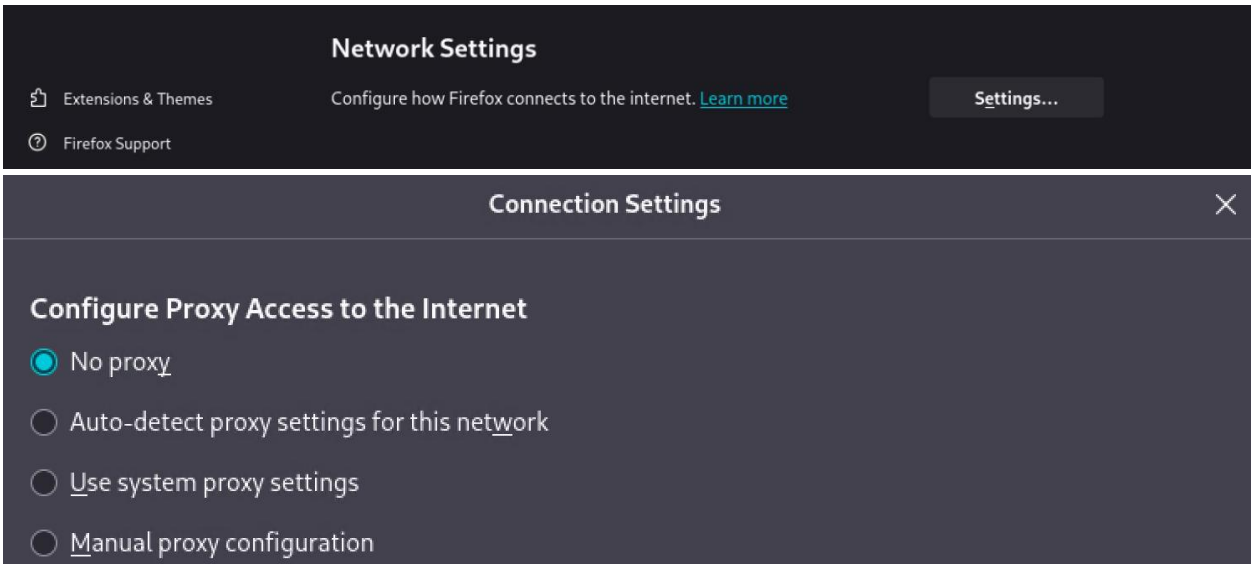




Under method, what does GET mean?

The GET means an HTTP request method used to retrieve data from a web server. It is used to request web pages or resources, and any parameters are included in the URL. In here, the GET request was observed when the browser requested pages from the DVWA application through Burp Suite.

16. To begin the audit, navigated to the Firefox browser and removed the Proxy setting as shown below. Then I can interact with the Web site.

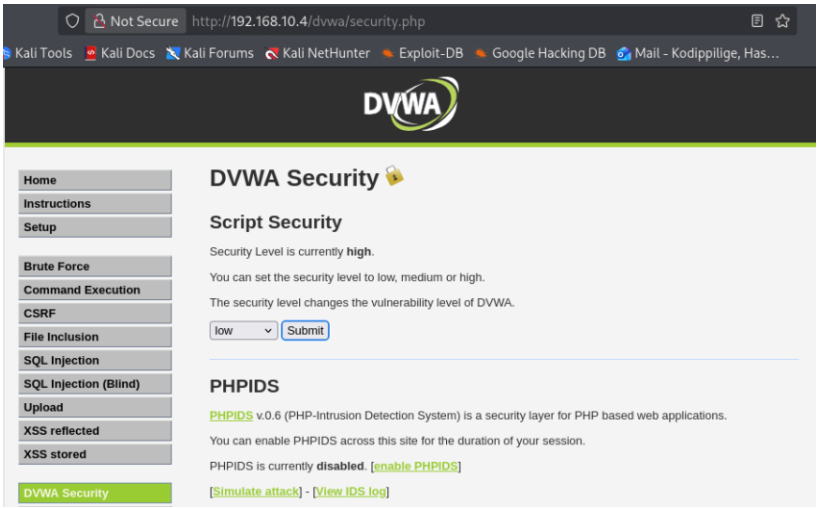


17. In Firefox, navigated to the Metasploitable VM URL 192.168.10.4 (Metasploitable VM must be turned ON).

18. Clicked on DVWA (Damn Vulnerable Web Application).

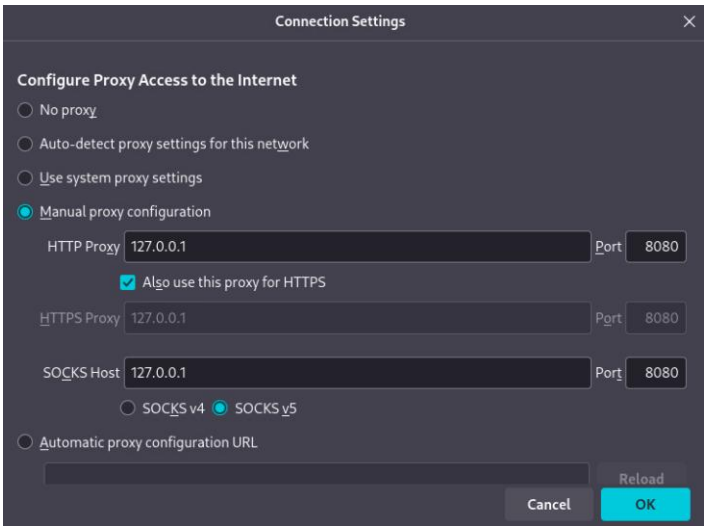
19. Logged in with username: *admin* and password: *password*.

20. Clicked *DVWA Security* and Set it to *low*. Then clicked *Submit*. This will allow vulnerability testing on the Web application.



21. Logged out of the web application, DVWA.

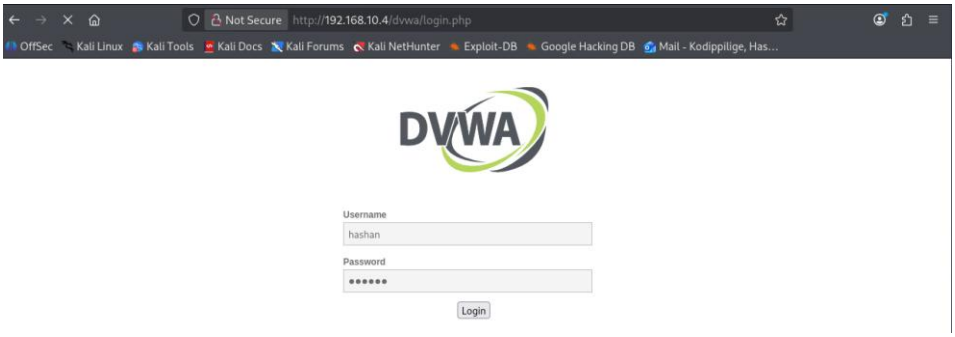
22. Changed the proxy settings again, so traffic is routed through Burp suite.



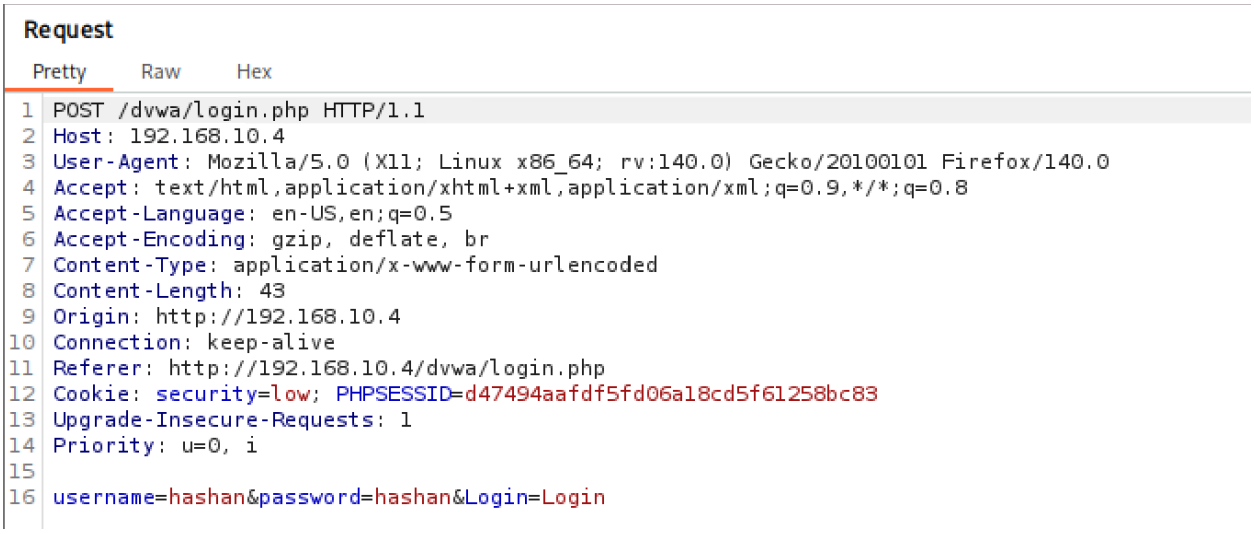
Test the DVWA for vulnerability to brute force attack

23. Opened Burp Suite. Ensure the URL for the Metasploitable VM is in Scope.

24. In Firefox, on the Sign in page, entered my first name for both user and password. Then clicked *Login*.



25. In Burp, clicked on the Intercept tab. Used time to select the transaction that captures my logged in attempt.



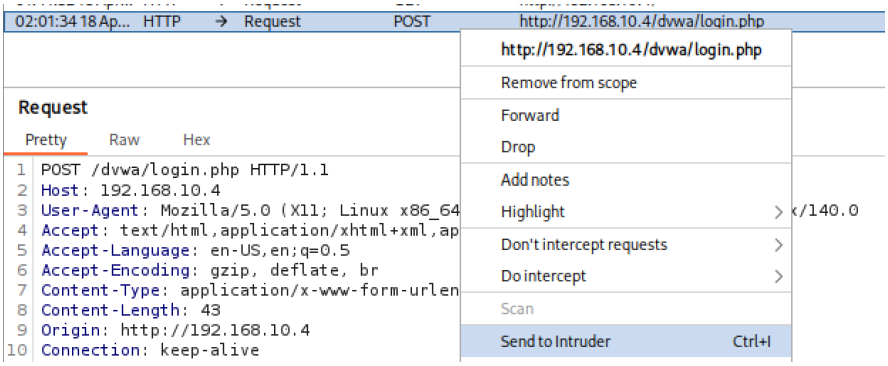
Describe the first three lines and the last line

The first line *POST /dvwa/login.php HTTP/1.1* shows that HTTP method Post used to send data to the server. The requested login page for DVWA indicates by */dvwa/login.php* and the protocol version used by HTTP/1.1

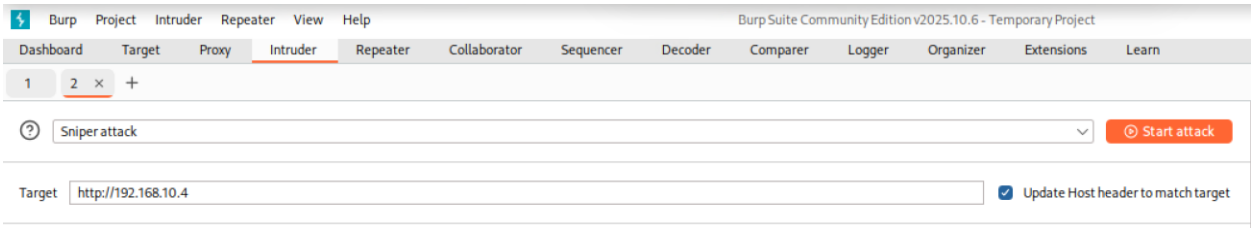
The second line *Host: 192.168.10.4* shows the request where it should be sent. In here, it is the Metasploitable VM hosting DVWA.

The third line identifies the client (browser and system) making the request. It includes browser type (Firefox) and operating system (Linux).

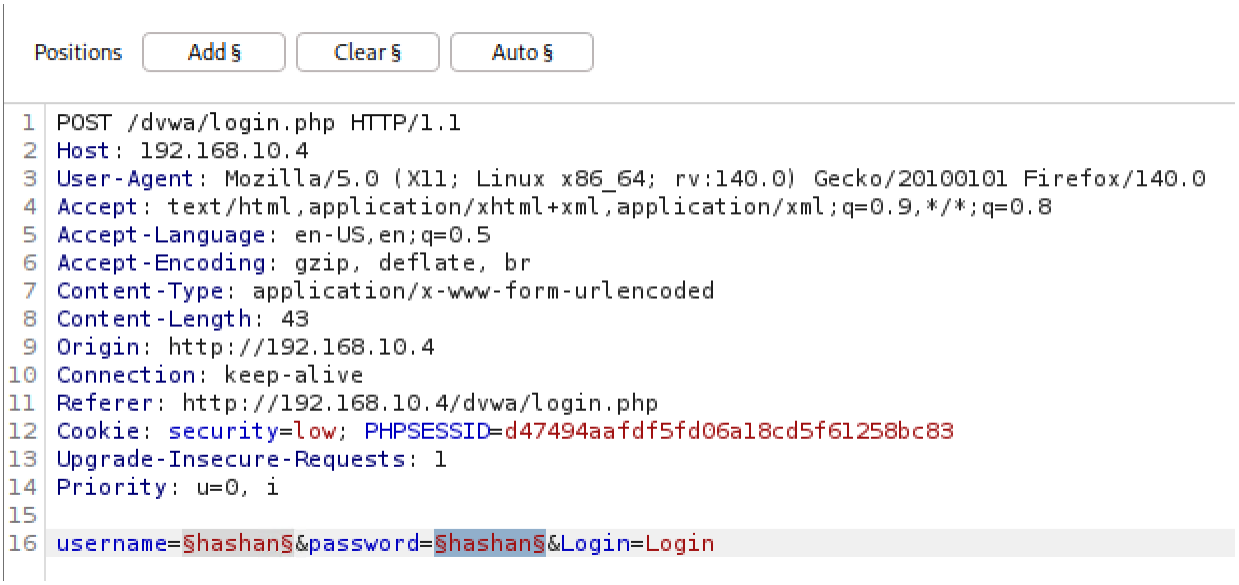
26. On the Intercept page, on the Request transaction that indicates login to DVWA, right-click on the *Request* and select 'Send to Intruder'.



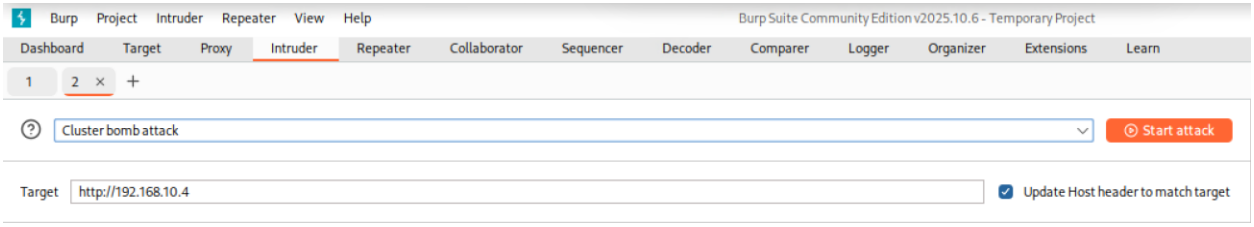
27. Clicked on *Intruder* tab. Target is the Metasploitable VM.



28. In the pane below, first double-clicked on the values for username. On '*Positions*' tab, clicked on the '*Add*' button. I did the same for the password, '*password*'. Clicked *Add*. The value for username and password enclosed in & symbol.



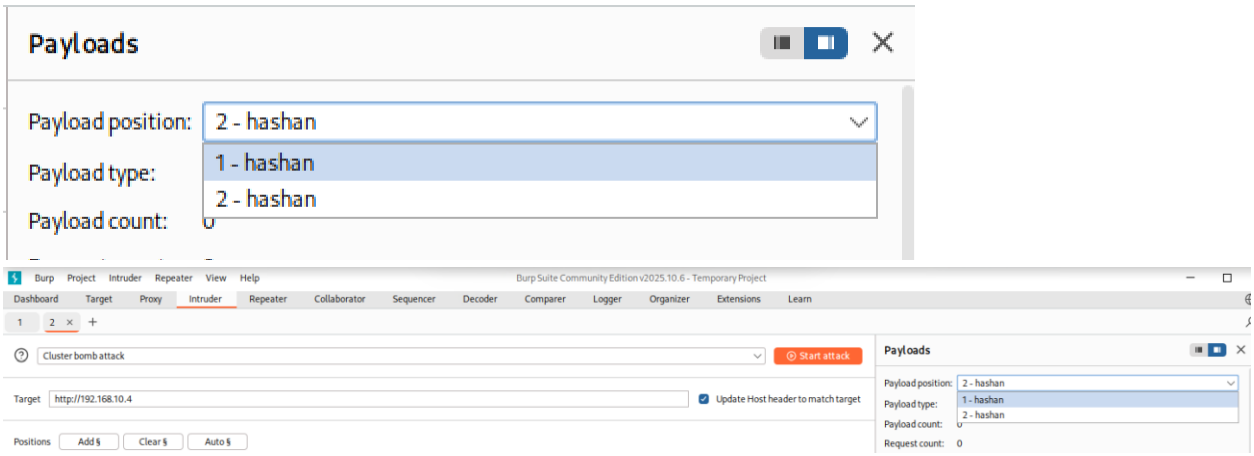
29. Changed attack type to *'Cluster bomb*' for brute force attack. Target is Metasploitable VM URL.



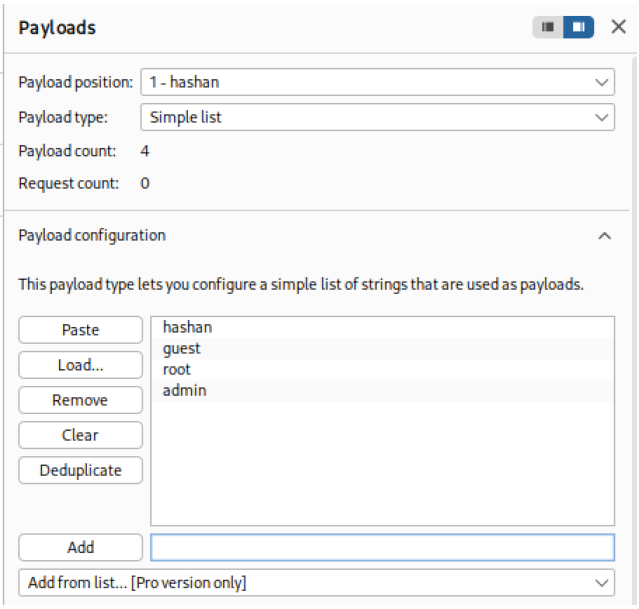
Step 3 – Creating Payloads

(creating different usernames and passwords as payloads)

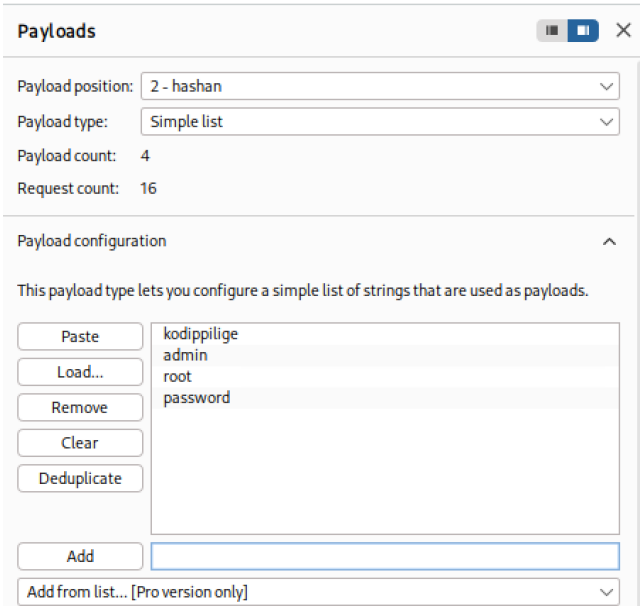
1. Verified that 1 & 2 for payload position on the right (1 is for username and 2 is for password).



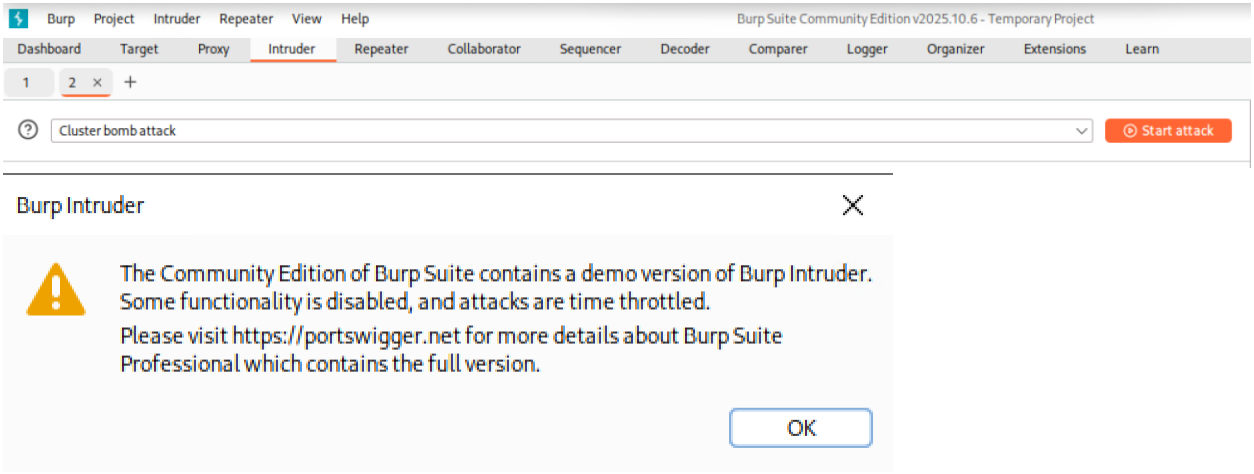
2. Under Payload Configuration, I added words for testing user name and password combination. Under Payload Configuration, entered *'hashan'*, 'guest', 'root', and 'admin' for user name and clicked '*Add*'.



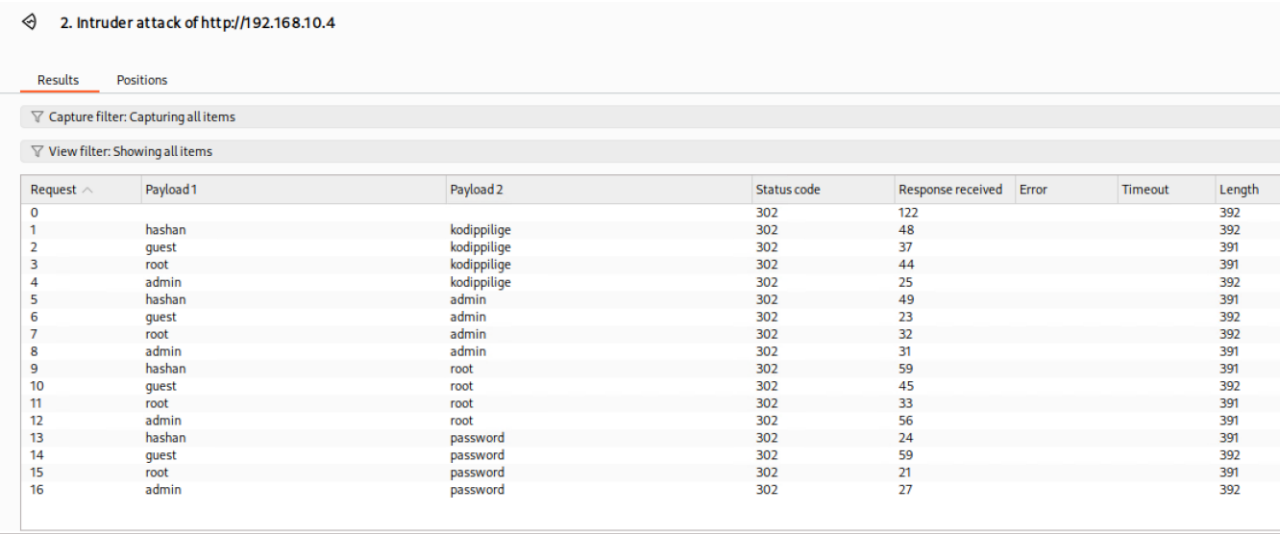
3. Did the same for passwords. Under payloads, selected '*2*' in payload position. Added entries *'kodippilige'*, 'admin', 'root', and 'password' in the Payload Configuration.



4. Clicked on *Start attack*.



5. The Intruder attack was executed using the cluster bomb method, which tested all possible combinations of the provided usernames and passwords. The results were displayed in a table format, showing each request along with its corresponding response.



2. Intruder attack of http://192.168.10.4

Results Positions

Capture filter: Capturing all items

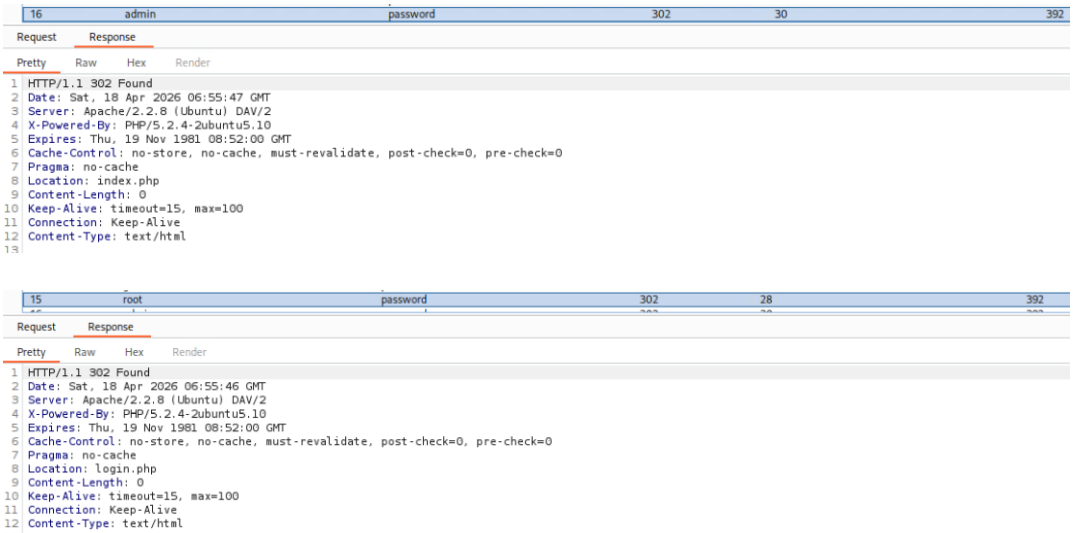
View filter: Showing all items

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length
0			302	122			392
1	hashan	kodippilige	302	48			392
2	guest	kodippilige	302	37			391
3	root	kodippilige	302	44			391
4	admin	kodippilige	302	25			392
5	hashan	admin	302	49			391
6	guest	admin	302	23			392
7	root	admin	302	32			392
8	admin	admin	302	31			391
9	hashan	root	302	59			391
10	guest	root	302	45			392
11	root	root	302	33			391
12	admin	root	302	56			391
13	hashan	password	302	24			391
14	guest	password	302	59			392
15	root	password	302	21			391
16	admin	password	302	27			392

All requests returned an HTTP status code of 302, indicating that the server redirected each login attempt.

In the attack results you obtained, can you tell which username and password combo was successful? Discuss.

It is difficult to determine the correct username and password combination, as all requests returned an HTTP status code of 302 and the response lengths were similar. Then I observed the *Response* tab, and it revealed that successful login attempts could be identified by examining the *Location* header. Failed login attempts redirected to *login.php*, while a successful login redirected to *index.php*. So, I identified that *admin* as the username and *password* as the password.



16 admin password 302 30 392

Request Response

Pretty Raw Hex Render

```
1 HTTP/1.1 302 Found
2 Date: Sat, 18 Apr 2026 06:55:47 GMT
3 Server: Apache/2.2.8 (Ubuntu) DAV/2
4 X-Powered-By: PHP/5.2.4-2ubuntu5.10
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
7 Pragma: no-cache
8 Location: index.php
9 Content-Length: 0
10 Keep-Alive: timeout=15, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html
```

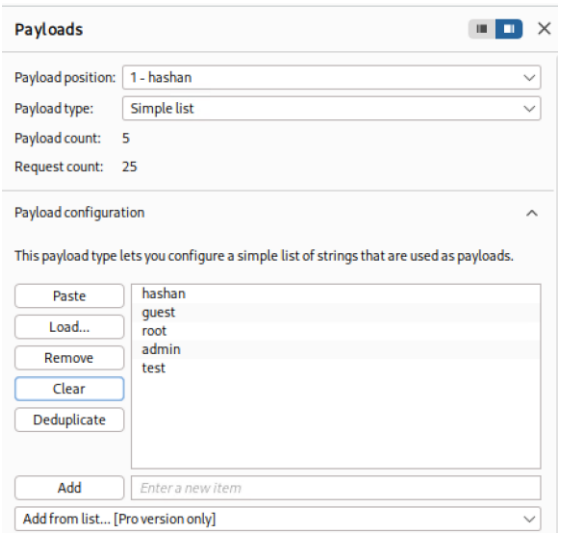
15 root password 302 28 392

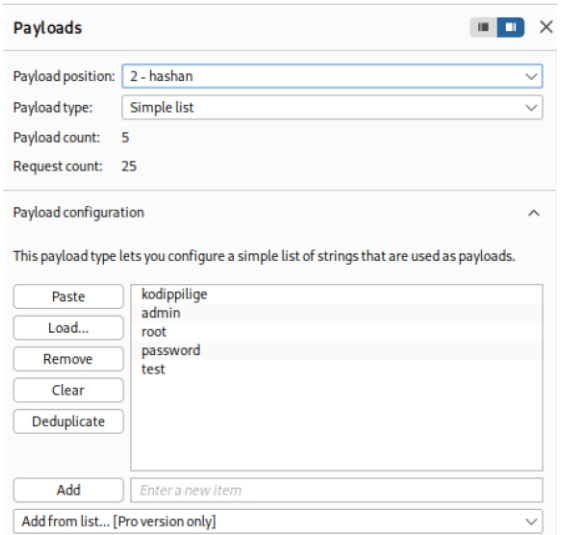
Request Response

Pretty Raw Hex Render

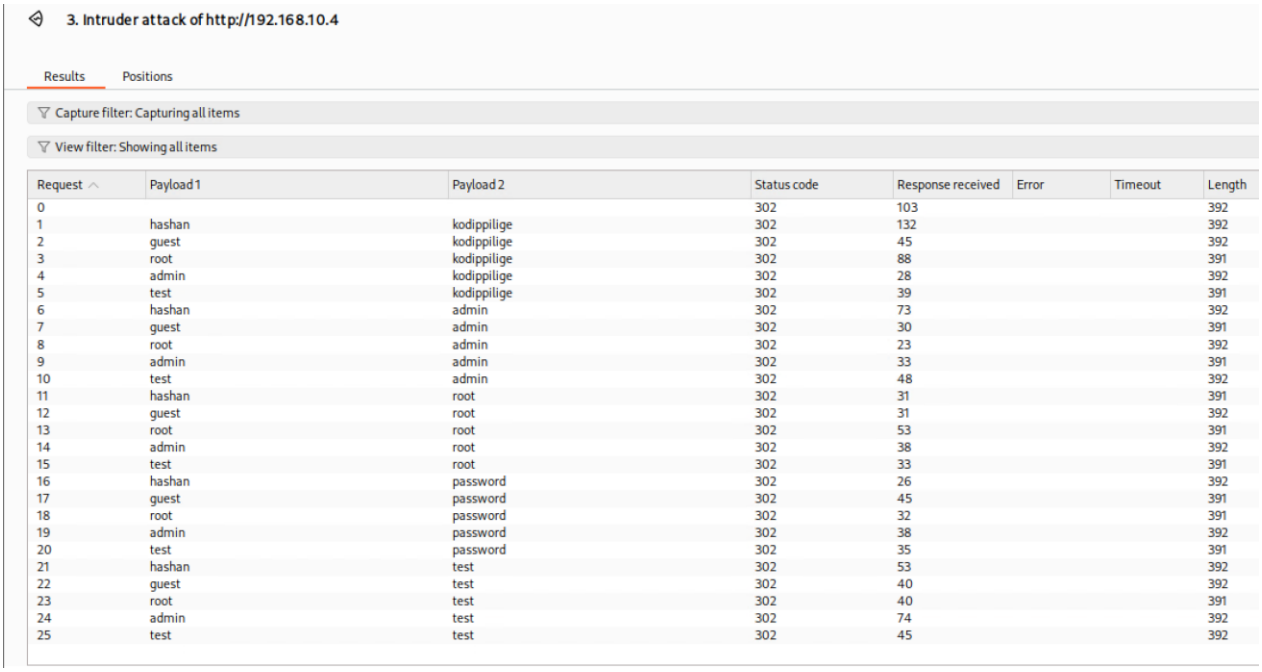
```
1 HTTP/1.1 302 Found
2 Date: Sat, 18 Apr 2026 06:55:46 GMT
3 Server: Apache/2.2.8 (Ubuntu) DAV/2
4 X-Powered-By: PHP/5.2.4-2ubuntu5.10
5 Expires: Thu, 19 Nov 1981 08:52:00 GMT
6 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
7 Pragma: no-cache
8 Location: login.php
9 Content-Length: 0
10 Keep-Alive: timeout=15, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html
```

6. Closed Intruder attack window results. Next added '*test*' word in payload option for both username and password. Then clicked *Start Attack*, again.



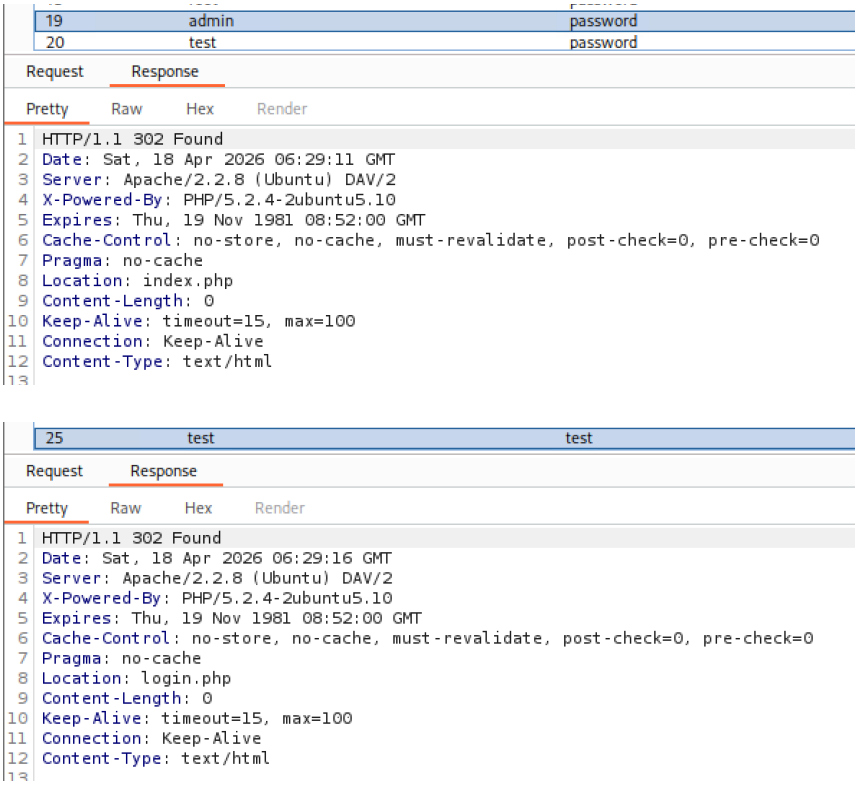


7. Observe the outcome of the attack.



Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length
0			302	103			392
1	hashan	kodippilige	302	132			392
2	guest	kodippilige	302	45			392
3	root	kodippilige	302	88			391
4	admin	kodippilige	302	28			392
5	test	kodippilige	302	39			391
6	hashan	admin	302	73			392
7	guest	admin	302	30			391
8	root	admin	302	23			392
9	admin	admin	302	33			391
10	test	admin	302	48			392
11	hashan	root	302	31			391
12	guest	root	302	31			392
13	root	root	302	53			391
14	admin	root	302	38			392
15	test	root	302	33			391
16	hashan	password	302	26			392
17	guest	password	302	45			391
18	root	password	302	32			391
19	admin	password	302	38			392
20	test	password	302	35			391
21	hashan	test	302	53			392
22	guest	test	302	40			392
23	root	test	302	40			391
24	admin	test	302	74			392
25	test	test	302	45			392

8. Studied the results by clicking on a line in the result and clicking the Response tab on bottom pane.



What signal indicates a login attempt succeeded? How can you tell?

Observed the response in Burp Suite *Location* header. Successful authentication was indicated by redirection to a different page (*index.php)* and failed attempts redirected back to the login page (*login.php)*. In here, change in redirection indicates that authentication was successful. The *Referer* field was same as all requests and did not indicate something when the login attempt succeeded.

Conclusion

This lab helps to cover the practical experience of how web application vulnerabilities can be identified through traffic interception and analysis using Burp Suite. It is possible to capture and examine HTTP traffic requests and responses, gaining insight into how authentication mechanisms function within a web application by configuring Burp Suite as a proxy. Brute force attacks can compromise weak authentication systems by using the burp suite with multiple username and password combinations.

The HTTP status code was insufficient to determine successful login attempts, as all responses returned a status code of 302 in this lab. It needs deeper analysis of the response headers, particularly the *Location* field which revealed the outcome of each login attempt. Successful

authentication was indicated by redirection to a different page and failed attempts redirected back to the login page.

This lab demonstrates the requirement of input validation, account lockout mechanisms, and secure session management to mitigate common web application vulnerabilities.

References

1. O'Leary, M. (2019). *Cyber operations: Building, defending, and attacking modern computer networks* (2nd ed.). Apress.
2. DigiNinja. (n.d.). *DVWA introduction*. Mintlify. https://mintlify.com/digininja/DVWA/introduction
3. Vaadata. (2024, January 15). *Introduction to Burp Suite, the tool dedicated to web application security*. https://www.vaadata.com/blog/introduction-to-burp-suite-the-tool-dedicated-to-web-application-security/
4. PortSwigger Ltd. (n.d.). *Burp Proxy*. https://portswigger.net/burp/documentation/desktop/tools/proxy
5. Raza, S. (2025, August 11). *Burp Suite intruder: Payload configuration guide (beginner-friendly)*. Medium. https://5r4z4.medium.com/burp-suite-intruder-payload-configuration-guide-beginner-friendly
6. Minnesota State University. (n.d.). Minnesota State Learning Management System.